



Department of Artificial Intelligence and Machine Learning

Experiment No.: 6

To implement Skip List

Name: Archit Vaishnav

Roll no. A121

SAP: 60017240174

Aim

To implement a Skip List and perform insertion, searching, deletion, and traversal operations using probabilistically generated levels.

Learning Objectives

- To understand the structure and working of a Skip List.
- To understand how multiple levels improve searching efficiency.
- To study the probabilistic method used for assigning levels to newly inserted elements.
- To implement insertion, searching, and deletion operations.
- To analyze the expected time complexity of Skip List operations.

Theory

A **Skip List** is a probabilistic data structure used to store elements in sorted order. It provides efficient searching, insertion, and deletion operations.

A Skip List consists of multiple levels of linked lists. The lowest level contains all the elements, while the higher levels contain only selected elements. These higher levels act as express lanes and allow the search operation to skip several elements at a time.

The structure can be visualized as follows:

Level 3: HEAD -----> 40 -----> NIL

Level 2: HEAD -----> 20 -----> 40 -----> NIL

Level 1: HEAD --> 10 -> 20 -----> 40 -> 50 --> NIL

Level 0: HEAD --> 10 -> 20 -> 30 -> 40 -> 50 --> NIL



Searching starts from the highest level. The algorithm moves forward while the next element is smaller than the required key. When further movement is not possible, it moves down one level.

This process continues until the required element is found or the lowest level is reached.

Level Assignment in Skip List

The level of a newly inserted node is generally determined **randomly**.

A probability p is selected, commonly

$$p = 0.5$$

Every node is always inserted at **Level 0**. The node is promoted to higher levels based on repeated random trials.

For example,

$$P(\text{Level} \geq 1) = \frac{1}{2}$$

$$P(\text{Level} \geq 2) = \frac{1}{4}$$

$$P(\text{Level} \geq 3) = \frac{1}{8}$$

Thus, the number of nodes decreases approximately by half as we move towards higher levels.

The maximum level can be selected based on the expected number of elements. A commonly used estimate is

$$L_{\max} \approx \log_{\frac{1}{p}} n$$

where n is the expected number of elements.

For $p = 0.5$,

$$L_{\max} \approx \log_2 n$$

Random Level Generation

A simple random level generation procedure is:

level = 0



```
while random() < p and level < MAX_LEVEL:
```

```
    level = level + 1
```

```
return level
```

For example, suppose the generated random numbers are:

0.31, 0.42, 0.76

For $p = 0.5$:

- $0.31 < 0.5 \Rightarrow$ promote to Level 1.
- $0.42 < 0.5 \Rightarrow$ promote to Level 2.
- $0.76 > 0.5 \Rightarrow$ stop.

Therefore, the new node is assigned **Level 2** and participates in Levels 0, 1, and 2.

Algorithm for Searching

1. Start from the header node at the highest level.
2. Compare the next node's key with the search key.
3. If the next key is smaller than the search key, move forward at the same level.
4. If the next key is greater than or equal to the search key, move down one level.
5. Repeat until Level 0 is reached.
6. If the next node contains the required key, report that the element is found.
7. Otherwise, report that the element is not present.

Algorithm for Insertion

1. Start searching from the highest level.
2. Maintain an update[] array to store the predecessor of the new node at each level.
3. Move forward while the next node contains a key smaller than the key to be inserted.
4. Move downward until Level 0 is reached.
5. Check whether the key is already present.



6. Generate a random level for the new node.
7. Create the new node with the generated level.
8. Update the forward pointers using the nodes stored in the update[] array.

Algorithm for Deletion

1. Start from the highest level of the Skip List.
2. Locate the predecessors of the node to be deleted and store them in the update[] array.
3. Move to Level 0 and check whether the required node exists.
4. If the node exists, modify all forward pointers that point to the node.
5. Delete the node.
6. If the highest level becomes empty, reduce the current maximum level.

Illustrative Example

Consider inserting the following elements:

10, 20, 30, 40, 50

Suppose the randomly generated levels are:

Element	Generated Level
10	1
20	2
30	0
40	3
50	1

The resulting Skip List is:

Level 3: HEAD -----> 40 -----> NIL

Level 2: HEAD -----> 20 -----> 40 -----> NIL

Level 1: HEAD --> 10 -> 20 -----> 40 -> 50 --> NIL

Level 0: HEAD --> 10 -> 20 -> 30 -> 40 -> 50 --> NIL



Example: Searching for 30

Suppose the Skip List is searched for key 30.

The search starts from the highest level.

Level 3:

HEAD -> 40

40 > 30

Move DOWN

Level 2:

HEAD -> 20

20 < 30

Move RIGHT to 40

40 -> 40

40 > 30

Move DOWN

Level 1:

20 -> 40

40 > 30

Move DOWN

Level 0:

20 -> 30

30 = Search Key

Element Found

Thus, instead of sequentially examining every node, the higher levels help the search skip unnecessary nodes.

Program

Write a program to implement a Skip List supporting the following operations:

1. Insert an element
2. Search for an element



3. Delete an element
4. Display the Skip List level by level

Expected Output

----- Skip List Operations -----

1. Insert

2. Search

3. Delete

4. Display

5. Exit

Enter choice: 1

Enter element: 10

Element inserted successfully.

Enter choice: 1

Enter element: 20

Element inserted successfully.

Enter choice: 1

Enter element: 30

Element inserted successfully.

Enter choice: 1

Enter element: 40

Element inserted successfully.

Enter choice: 1

Enter element: 50

Element inserted successfully.

Enter choice: 4

Skip List:

Level 3: 40

Level 2: 20 40

Level 1: 10 20 40 50

Level 0: 10 20 30 40 50

Enter choice: 2

Enter element to search: 30

Element 30 found.

Enter choice: 3

Enter element to delete: 30

Element 30 deleted successfully.



Level 0: 10 20 40 50

Code

```
main.py > ...
1  # node definition
2  import random
3  class Node:
4      # Every node in a skip list holds a value and an array of pointers (forward references) to the next nodes at different levels.
5      def __init__(self, val, level):
6          self.val = val
7          # forward is an array of pointers to the next node at each level
8          self.forward = [None] * (level + 1)
9
10 # definition of the skip list structure with p = 0.5 and max level = 16
11 class SkipList:
12     def __init__(self, max_level=16, p=0.5):
13         self.max_level = max_level
14         self.p = p
15         self.head = Node(None, self.max_level)
16         self.level = 0 # Current highest level in the skip list
17
18     def random_level(self):
19         lvl = 0
20         while random.random() < self.p and lvl < self.max_level:
21             lvl += 1
22         return lvl
23
24     def search(self, target):
25         curr = self.head
26         # Traverse downwards from the highest current level to level 0
27         for i in range(self.level, -1, -1):
28             while curr.forward[i] and curr.forward[i].val < target:
29                 curr = curr.forward[i]
30
31         curr = curr.forward[0]
32         if curr and curr.val == target:
33             return True
34         return False
35
36     # insert function
37     def insert(self, val):
38         update = [None] * (self.max_level + 1)
39         curr = self.head
40
41         # Find position to insert across all levels
42         for i in range(self.level, -1, -1):
43             while curr.forward[i] and curr.forward[i].val < val:
44                 curr = curr.forward[i]
45             update[i] = curr
46         curr = curr.forward[0]
47
48         # If value already exists, we can skip or update (here we skip duplicates)
49         if curr is None or curr.val != val:
```

```
main.py > ...
10 class SkipList:
36     def insert(self, val):
47         # If value already exists, we can skip or update (here we skip duplicates)
48         if curr is None or curr.val != val:
49             r_level = self.random_level()
50
51         # If random level is greater than current list level, initialize update pointers
52         if r_level > self.level:
53             for i in range(self.level + 1, r_level + 1):
54                 update[i] = self.head
55             self.level = r_level
56
57         # Create new node and insert it by updating pointers
58         new_node = Node(val, r_level)
59         for i in range(r_level + 1):
60             new_node.forward[i] = update[i].forward[i]
61             update[i].forward[i] = new_node
62
63     # function to delete a node from the skip list
64     def delete(self, val):
65         update = [None] * (self.max_level + 1)
66         curr = self.head
67
68         # Find the node and record update pointers
69         for i in range(self.level, -1, -1):
70             while curr.forward[i] and curr.forward[i].val < val:
71                 curr = curr.forward[i]
72             update[i] = curr
73
74         curr = curr.forward[0]
75
76         # If node exists, remove references
77         if curr and curr.val == val:
78             for i in range(self.level + 1):
79                 if update[i].forward[i] != curr:
80                     break
81             update[i].forward[i] = curr.forward[i]
82
83         # Drop the list level if top levels are now empty
84         while self.level > 0 and self.head.forward[self.level] is None:
85             self.level -= 1
86         return True
87         return False
88
89     # function to display
90     def display(self):
91         print("\n--- Skip List ---")
92         for i in range(self.level, -1, -1):
```



```
main.py > ...
10 class SkipList:
11     def __init__(self):
12         self.level = 1
13         self.head = Node(0)
14         self.head.forward[0] = None
15
16     def display(self):
17         print("\n--- Skip List ---")
18         for i in range(self.level, -1, -1):
19             curr = self.head.forward[i]
20             line = f"Level {i}: "
21             while curr:
22                 line += f"{curr.val} -> "
23                 curr = curr.forward[i]
24             print(line + "None")
25         print("-----\n")
26
27 def main():
28     skiplist = SkipList()
29     while True:
30         print("---- Skip List Operations----")
31         print("1. Insert")
32         print("2. Search")
33         print("3. Delete")
34         print("4. Display")
35         print("5. Exit")
36         try:
37             choice = int(input("Enter choice: "))
38         except ValueError:
39             print("Please enter a valid number.")
40             continue
41
42         if choice == 1:
43             val = int(input("Enter element: "))
44             skiplist.insert(val)
45             print("Element inserted successfully.")
46         elif choice == 2:
47             val = int(input("Enter element to search: "))
48             if skiplist.search(val):
49                 print(f"Element {val} found.")
50             else:
51                 print(f"Element {val} not found.")
52         elif choice == 3:
53             val = int(input("Enter element to delete: "))
54             if skiplist.delete(val):
55                 print(f"Element {val} deleted successfully.")
56             else:
57                 print(f"Element {val} not found in the skip list.")
58         elif choice == 4:
59             skiplist.display()
60         elif choice == 5:
61             break
62     else:
63         pass
```

```
main.py > ...
103 def main():
104     print("---- Skip List Operations----")
105     print("1. Insert")
106     print("2. Search")
107     print("3. Delete")
108     print("4. Display")
109     print("5. Exit")
110     try:
111         choice = int(input("Enter choice: "))
112     except ValueError:
113         print("Please enter a valid number.")
114         continue
115
116     if choice == 1:
117         val = int(input("Enter element: "))
118         skiplist.insert(val)
119         print("Element inserted successfully.")
120     elif choice == 2:
121         val = int(input("Enter element to search: "))
122         if skiplist.search(val):
123             print(f"Element {val} found.")
124         else:
125             print(f"Element {val} not found.")
126     elif choice == 3:
127         val = int(input("Enter element to delete: "))
128         if skiplist.delete(val):
129             print(f"Element {val} deleted successfully.")
130         else:
131             print(f"Element {val} not found in the skip list.")
132     elif choice == 4:
133         skiplist.display()
134     elif choice == 5:
135         break
136     else:
137         print("Invalid choice. Try again.")
138
139 if __name__ == "__main__":
140     main()
```



Output

```
C:\Users\djsce.student\Desktop\A121_ADSA\ADSA_Lab\Exp06>python main.py
—— Skip List Operations——
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter element: 23
Element inserted successfully.
—— Skip List Operations——
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 10
Invalid choice. Try again.
—— Skip List Operations——
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter element: 3
Element inserted successfully.
—— Skip List Operations——
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter element: 22
Element inserted successfully.
—— Skip List Operations——
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4

—— Skip List ——
Level 3: 22 → None
Level 2: 22 → None
Level 1: 22 → None
```



Problem Statement

3417. Zigzag Grid Traversal With Skip

Easy Topics Companies

You are given an $m \times n$ 2D array `grid` of **positive** integers.

Your task is to traverse `grid` in a **zigzag** pattern while skipping every **alternate** cell.

Zigzag pattern traversal is defined as following the below actions:

- Start at the top-left cell $(0, 0)$.
- Move *right* within a row until the end of the row is reached.
- Drop down to the next row, then traverse *left* until the beginning of the row is reached.
- Continue **alternating** between right and left traversal until every row has been traversed.

Note that you **must skip** every *alternate* cell during the traversal.

Return an array of integers `result` containing, **in order**, the value of the cells visited during the zigzag traversal with skips.

Example 1:

Input: `grid = [[1,2],[3,4]]`

Output: `[1,4]`

Explanation:

The screenshot displays a LeetCode submission for the problem '3417. Zigzag Grid Traversal With Skip'. The submission is in Python3 and has been accepted. The runtime is 3 ms and memory usage is 19.48 MB. The test results show 'Accepted' for all three cases.

Runtime: 3 ms, Beats: 45.42%, Memory: 19.48 MB, Beats: 77.08%

Code:

```
class Solution:
    def zigzagTraverse(self, grid: List[List[int]]) -> List[int]:
        result = []
        for i in range(len(grid)):
            if i % 2 == 0:
                for j in range(len(grid[i])):
                    result.append(grid[i][j])
            else:
                for j in range(len(grid[i]) - 1, -1, -1):
                    result.append(grid[i][j])
        return result
```

Test Results: Accepted (Runtime: 0 ms)

Case 1: Input: `grid = [[1,2],[3,4]]`, Output: `[1,4]`, Expected: `[1,4]`



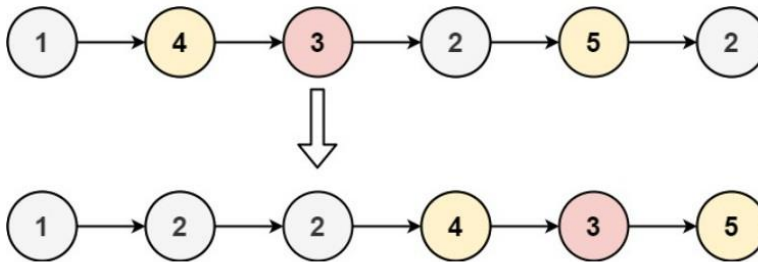
86. Partition List

Medium Topics Companies

Given the `head` of a linked list and a value `x`, partition it such that all nodes **less than** `x` come before nodes **greater** **equal** to `x`.

You should **preserve** the original relative order of the nodes in each of the two partitions.

Example 1:



Input: head = [1,4,3,2,5,2], x = 3

Output: [1,2,2,4,3,5]

Example 2:

Input: head = [2,1], x = 2

Output: [1,2]

Accepted 169 / 169 testcases passed
ArchitVaidya submitted at Aug 24, 2026 13:29

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one place.

Runtime: 0 ms, Beats 100.00%, Memory: 19.36 MB, Speed: 38.48%

Code: Python3

```
1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def partition(self, head: Optional[ListNode], x: int) -> Optional[ListNode]:
8         less_head = ListNode()
9         greater_head = ListNode()
10        less_curr = less_head
11        greater_curr = greater_head
12        curr = head
13        while curr:
14            if curr.val < x:
15                less_curr.next = curr
16                less_curr = less_curr.next
17            else:
18                greater_curr.next = curr
19                greater_curr = greater_curr.next
20        less_curr.next = greater_head.next
21        return less_head
```

Testcase: Runtime: 0 ms

Case 1 Case 2

Input: head = [1,4,3,2,5,2], x = 3

Output: [1,2,2,4,3,5]

Expected: [1,2,2,4,3,5]



Post-Lab Questions

1. What is a Skip List? How does it differ from a normal linked list?
2. What is the purpose of maintaining multiple levels in a Skip List?
3. How is the level of a newly inserted node determined?
4. Why is the level assignment in a Skip List considered probabilistic?
5. If $p = 0.5$, what is the probability that a node reaches Level 3?
6. Explain the role of the `update[]` array during insertion.
7. Why does searching begin from the highest level rather than Level 0?
8. What is the expected time complexity of searching in a Skip List?
9. What is the worst-case time complexity of searching in a Skip List? Under what condition can this occur?
10. Construct a Skip List for the keys

15, 25, 35, 45, 55, 65

assuming their generated levels are

1, 3, 0, 2, 1, 0

respectively. Show the Skip List at every level.

References

1. William Pugh, "Skip Lists: A Probabilistic Alternative to Balanced Trees," *Communications of the ACM*, Vol. 33, No. 6, 1990.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis*, Pearson Education.